Day12 Hands On Tasks - ANSI SQL USING MS-SQL Server

Level-1 and 2 Problem:

Create Database **EventDb** and create tables as per schema given below.

Requirements:

1.Add entity class **UserInfo** and add public properties:

Property Name	Type	Constraint
EmailId	varchar	**PK**
UserName	varchar	Not Null, Length: 1 to 50 char.
Role	varchar	Not Null, (Admin,Participant)-CHECK
Password	varchar	Not Null Length: 6 to 20 char

2. Add entity class **EventDetails** and add below public properties.

Property Name	Type	Constraint
EventId	int	**PK**
EventName	varchar	Not Null, Length: 1 to 50 char.
EventCategory	varchar	Not Null, Length: 1 to 50 char.
EventDate	datetime	Not Null
Description	varchar	Nullable
Status	varchar	Active or In-Active, CHECK

4. Add entity class **SpeakersDetails** and add below public properties.

Property Name	Type	Constraints
SpeakerId	int	**PK**
SpeakerName	varchar	Not Null, Length: 1 to 50 char.

5. Add entity class **SessionInfo** and add below public properties.

Property Name	Type	Constraints
SessionId	int	**PK**
EventId	int	Not Null **(FK)**
SessionTitle	varchar	Not Null, Length: 1 to 50 char.
SpeakerId	int	Not Null **(FK)**
Description	varchar	Nullable
SessionStart	datetime	Not Null (Time Slot)
SessionEnd	datetime	Not Null (Time Slot)
SessionUrl	varchar	

6. Add entity class **ParticipantEventDetails** and add below public properties.

Property Name	Type	Constraints
Id	int	**(PK)**
ParticipantEmailId	varchar	Not Null **(FK)**
EventId	int	Not Null **(FK)**
SessionId	Int	Not Null **(FK)**
IsAttended	bit	Yes or No, CHECK

Technical Constraints

- Use SQL Server.
- Use appropriate data types (INT, VARCHAR, DATE, DECIMAL).
- Follow normalization up to 2NF.
- Use ANSI SQL syntax only.

Expectations:

- Proper table structure with relationships.
- Valid constraints implementation.
- Correct data insertion and retrieval queries.

Learning Outcome

You will understand database structure, basic normalization, constraints, and ANSI SQL queries.

1. Creating Database: EventDb

```
CREATE DATABASE EventDb;
```

2. Using the Database:

```
USE EventDb;
```

3. Creating Table: UserInfo

```sql
CREATE TABLE UserInfo (
    EmailId VARCHAR(100) PRIMARY KEY,
    UserName VARCHAR(50) NOT NULL,
    Role VARCHAR(20) NOT NULL,
    Password VARCHAR(20) NOT NULL,

    CONSTRAINT CK_UserName_Length
        CHECK (LEN(UserName) BETWEEN 1 AND 50),

    CONSTRAINT CK_Role
        CHECK (Role IN ('Admin', 'Participant')),

    CONSTRAINT CK_Password_Length
        CHECK (LEN(Password) BETWEEN 6 AND 20)
);
```

4. Creating Table: EventDetails

```sql
CREATE TABLE EventDetails (
    EventId INT PRIMARY KEY,
    EventName VARCHAR(50) NOT NULL,
    EventCategory VARCHAR(50) NOT NULL,
    EventDate DATETIME NOT NULL,
    Description VARCHAR(255) NULL,
    Status VARCHAR(20) NOT NULL,

    CONSTRAINT CK_Status CHECK (Status IN('Active','In-Active'))
);
```

5. Creating Table: SpeakersDetails

```sql
CREATE TABLE SpeakersDetails (
    SpeakerId INT PRIMARY KEY,
    SpeakerName VARCHAR(50) NOT NULL
);
```

6. Creating Table: SessionInfo

```sql
CREATE TABLE SessionInfo (
    SessionId INT PRIMARY KEY,
    EventId INT NOT NULL,
    SessionTitle VARCHAR(50) NOT NULL,
    SpeakerId INT NOT NULL,
    Description VARCHAR(255) NULL,
    SessionStart DATETIME NOT NULL,
    SessionEnd DATETIME NOT NULL,
    SessionUrl VARCHAR(255),

    FOREIGN KEY (EventId) REFERENCES EventDetails(EventId),
    FOREIGN KEY (SpeakerId) REFERENCES SpeakersDetails(SpeakerId)
);
```

7. Creating Table: ParticipantEventDetails

```sql
CREATE TABLE ParticipantEventDetails (
    Id INT PRIMARY KEY,
    ParticipantEmailId VARCHAR(100) NOT NULL,
    EventId INT NOT NULL,
    SessionId INT NOT NULL,
    IsAttended BIT NOT NULL,

    FOREIGN KEY (ParticipantEmailId) REFERENCES UserInfo(EmailId),
    FOREIGN KEY (EventId) REFERENCES EventDetails(EventId),
    FOREIGN KEY (SessionId) REFERENCES SessionInfo(SessionId)
);
```

8. Data Insertion:

- Insert into **UserInfo**

```sql
--- Inserting data into UserInfo Table
INSERT INTO UserInfo VALUES ('admin@test.com','AdminUser','Admin','admin123');
INSERT INTO UserInfo VALUES ('user1@test.com','Ravi','Participant','ravi123');
```

- Insert into **EventDetails**

```sql
--- Inserting Data into EventDetails Table
INSERT INTO EventDetails VALUES (1, 'Tech Event', 'IT', GETDATE(), 'Annual Event', 'Active');
```

- Insert into **SpeakersDetails**

```sql
--- Inserting Data into SpeakerDetails Table
INSERT INTO SpeakersDetails VALUES (1,'Dr Sharma');
```

- Insert into **SessionInfo**

```sql
--- Inserting Data into SessionInfo Table
INSERT INTO SessionInfo VALUES (1, 1, 'AI Future', 1, NULL, GETDATE(), DATEADD(HOUR, 1, GETDATE()), NULL);
```

- Insert into **ParticipantEventDetails**

```sql
--- Inserting Data into ParticipantEventDetails Table
INSERT INTO ParticipantEventDetails VALUES (1, 'user1@test.com', 1, 1, 1);
```

9. Data Retrieval Queries

- **View All Users**

```
SELECT * FROM UserInfo;
```

Results | Messages

	EmailId	UserName	Role	Password
1	admin@test.com	AdminUser	Admin	admin123
2	user1@test.com	Ravi	Participant	ravi123

- **EventDetails**

```
SELECT * FROM EventDetails;
```

Results | Messages

	EventId	EventName	EventCategory	EventDate	Description	Status
1	1	Tech Event	IT	2026-03-03 18:32:14.150	Annual Event	Active

- **SpeakersDetails**

```
SELECT * FROM SpeakersDetails;
```

Results | Messages

	SpeakerId	SpeakerName
1	1	Dr Sharma

- **SessionInfo**

```
SELECT * FROM SessionInfo;
```

Results | Messages

	SessionId	EventId	SessionTitle	SpeakerId	Description	SessionStart	SessionEnd	SessionUrl
1	1	1	AI Future	1	NULL	2026-03-03 18:35:58.960	2026-03-03 19:35:58.960	NULL

- **ParticipantEventDetails**

```
SELECT * FROM ParticipantEventDetails;
```

Results | Messages

	Id	ParticipantEmailId	EventId	SessionId	IsAttended
1	1	user1@test.com	1	1	1